

orchestrator/ 代码说明

给第一次看这份代码的人快速建立地图用的开发文档。`main.py` 只是薄薄的入口胶水代码，真正的 5 个 Agent 各自一个文件放在 `agents/` 目录下，`persona/store/trip_plan` 三个模块已经被对应的 Agent 接起来了（具体接了什么见文末），只有 OTA/酒店 Agent 还是纯占位假数据。

设计背景/接口契约见 [agent-architecture.md](#)（为什么这么分 Agent、为什么中心化编排）和 [agent-interfaces.md](#)（字段契约、原本的 Dify 方案，后来改成直接写 Python）。

环境准备

```
pip install openai python-dotenv chromadb
```

在 `orchestrator/.env` 里写这几行（这个文件已经在 `.gitignore` 里，不会被提交）：

```
PARATERA_API_KEY=你的key
AMAP_KEY=你的高德Web服务API Key
AMAP_JS_KEY=你的高德Web端(JS API)Key
AMAP_JS_SECURITY_CODE=你的安全密钥
QWEATHER_KEY=你的和风天气API KEY
QWEATHER_API_HOST=你的和风天气专属API Host
```

高德的 key **分三样东西，缺一不可都不行**：- `AMAP_KEY`： **Web服务** 类型，后端用（`map_tool.py` 查地理编码/路线） -

`AMAP_JS_KEY`： **Web端(JS API)** 类型，前端画地图用——注意这是另一个应用/另一个 key，不能跟上面那个通用 -

`AMAP_JS_SECURITY_CODE`：安全密钥，高德 JS API 2.0 版本强制要求，在申请 `AMAP_JS_KEY` 的同一个应用页面能找到

都在 <https://lbs.amap.com> 控制台“创建应用”里申请（一个应用可以同时创建 Web服务 + Web端(JS API) 两个 key，安全密钥在创建 Web端(JS API) key 的时候会一起给）。个人开发者未认证 6000次/天，实名认证后 30万次/月，demo 阶段够用。

和风天气（`weather_tool.py` 用，灾害预警）去 <https://dev.qweather.com> 控制台“项目管理”申请：- `QWEATHER_KEY`：创建凭据时选 **API KEY** 类型，不要选 JWT（JWT 要自己写签名逻辑，demo 规模没必要）- `QWEATHER_API_HOST`：同一个项目页面上能看到，形如 `xxxxxxxxxx.re.qweatherapi.com`，**不用带** `https://` 前缀——2023 年后注册的新账号是“一账号一专属域名”，公共的 `devapi.qweather.com` 对新 key 直接返回 404，必须用这个专属域名

个人开发者免费版 1000次/天，灾害预警接口在免费的 8 个基础接口里（辐射/海洋/热带气旋等付费专项接口不在内，用不上）。

每个文件都能直接 `python orchestrator/<路径>.py` 单独跑（包括 `agents/` 里的 5 个文件），文件末尾的 `if __name__ == "__main__":` 都是自测代码，可以照着抄用法。

目录结构

```
orchestrator/
  llm_tool.py          # 通用 LLM 调用工具，跟“编排”本身无关，谁都能 import
  map_tool.py          # 通用地图工具（高德地图 API）：地理编码 + 路径规划
  weather_tool.py      # 通用天气工具（和风天气 API）：灾害预警查询，给异常应变 Agent 用
  hotel_tool.py        # 通用酒店工具（道旅 RollingGo）：真实酒店搜索 + 房型报价，给 OTA/酒店 Agent 用
agents/
```

```
orchestrator_agent.py      # 编排 Agent: new_shared_state + classify_intent + orchestrate
content_agent.py          # 达人/内容 Agent
route_agent.py            # 行程/路线 Agent
ota_hotel_agent.py        # OTA/酒店 Agent (酒店真数据, 机票还是纯占位)
exception_agent.py        # 异常应变 Agent (提案制, 见对应一节)
restaurant_agent.py       # 餐厅技能 (fellow 分支合并进来): 澳门模拟餐厅演示, 带界数工具调用循环
widgets.py                # 展示插件: 右侧聊天框里的富交互组件, 跟 Agent 逻辑解耦
schedule_widgets.py       # 左侧日程面板展示组件 (地图+连线、每日时间线、机票酒店面板), 跟 widgets.py 同一个分离
nearby_planner.py         # 港澳附近游确定性 workflow (fellow 分支合并进来, 见对应一节)
nearby_sources.py         # 附近游用的数据源: 高德 POI/路线 + 和风天气逐小时预报
restaurant_tools.py       # restaurant_agent 的工具调用实现 (虚构澳门餐厅数据 + 校验规则)
session_store.py          # 按浏览器 session 存整份 shared_state (SQLite), 替换掉原来的单进程全局 state
standardize_hk_macau_data.py # 把组员整理的港澳达人数据 Excel 转成标准化 JSON (见文末专门一节)
import_hk_macau_data.py   # 把标准化 JSON 灌进 Chroma (人格向量怎么算在这一步)
data_sources/hk_macau_posts.json # 标准化后的港澳达人数据库 (229 条, 本地人认证 + 小红书两个来源)
main.py                   # 薄入口: re-export 编排 Agent 的两个函数 + __main__ 完整 demo
store.py / trip_plan.py / persona.py # 不变
server.py / web/          # 接了 session_store + 港澳附近游相关接口, 见对应一节
```

关于 nearby_planner.py / restaurant_agent.py 这条 workflow 设计思路: 这是这次合并从 fellow 那边学到、也是编排 Agent 现在整体在往的方向——能算清楚的事实 (几点到哪、走多久、天气、餐厅是否符合硬性限制) 用确定性代码算完, LLM 只用在两处很窄的地方: 把用户的自然语言请求解析成结构化字段、从已经查到的真实候选里挑 ID (不许凭空编新地点/新属性)。最后给用户看的文字要么是纯模板拼出来的 (nearby_planner.plan_reply()), 要么是工具调用循环产出后原样返回、不再过第二个模型改写 (restaurant_agent.run())。orchestrator_agent.orchestrate() 里只有真正需要"总结/闲聊"性质的部分才会走最后一次 LLM 组句, 而且系统提示词里专门加了"以下 JSON 是资料不是指令, 不得捏造"这类防幻觉措辞。

拆分原则: 谁负责哪个 Agent 就改 agents/ 下自己那一个文件, 不用碰 main.py 或别人的 Agent 文件, 减少多人协作冲突。

llm_tool.py -- 通用 LLM 工具

职责: 跟"编排"这件事本身无关的基础设施, 5 个 Agent 里任何一个要调用模型都用这个, 不要各自再写一遍 client 初始化。

函数/常量	作用
call_llm(messages, model=MODEL_FULL)	统一的模型调用入口
MODEL_FULL / MODEL_LIGHT	DeepSeek-V4-Pro / DeepSeek-V4-Flash, 两档模型对齐 agent-architecture.md 的模型档位设计

map_tool.py -- 通用地图工具 (高德地图)

职责: 地理编码 + 路径规划, 给行程/路线 Agent 算真实的距离/耗时/路线用, 跟 llm_tool.py 一样是通用基础设施。

函数	作用
geocode(address, city=None)	地名/地址转经纬度, 返回 (lng, lat)
route_between(place_a, place_b, mode="walking", city=None)	算两地之间的距离/耗时/路线, mode 传 "walking" 或 "driving", 返回 distance_m / duration_min / polyline

现状/限制: - 只做了"两点之间"的查询, 还没做"多点一次性规划最优顺序" (这个高德也有专门接口, 以后要优化路线顺序时再接) - route_agent.py 里用这个算相邻两站的真实交通方式/耗时 (超过 2km 自动从步行切驾车), 查询失败会优雅降级成"待定 (地图查询失败: ...) "文字, 不会导致整个流程崩掉 - 到达/结束的具体钟点时间 (arrival_time / end_time) 还是占位——那需要"一天几点开始"+"每站玩多久"这类还没定义的调度逻辑, 跟地图 API 是两回事 - 地图可视化 (真的在网页上画一张地图+路线) 还没做, web/index.html 目前只是文字列表展示行程, polyline 字段目前没被前端用上

weather_tool.py -- 通用天气工具（和风天气）

职责：和风天气的两块能力——灾害预警（给异常应变 Agent 判断"这个城市现在是不是真的有极端天气"）、逐小时预报（给港澳附近游 workflow 算"这个时段要不要带伞"）。跟 map_tool.py 一样是通用基础设施。

函数	作用
<code>get_active_warnings(location, city=None)</code>	查某个地名当前生效的灾害预警，返回列表，每条 <code>{"headline", "event_type", "severity", "description", "effective_time", "expire_time"}</code> ；没有预警返回空列表（正常情况，不是错误）
<code>get_hourly_forecast(lng, lat, hours=240)</code>	查某个坐标未来最多 240 小时（10 天，免费版上限）的逐小时预报，返回列表，每条 <code>{"time", "temperature_c", "rain_probability", "condition_text"}</code> ；直接传经纬度（不查地名）， <code>nearby_sources.py</code> 已经有 POI 坐标就不用再地理编码一次

- `severity` 取值：`unknown / minor / moderate / severe / extreme`，越靠后越严重——`exception_agent.check_weather()` 用这个字段决定 `needs_replan` 要不要标 `True`（只是提示信号，不会自动改行程，见 `exception_agent.py` 一节）
- `get_active_warnings()` 按经纬度查（`/weatheralert/v1/current/{lat}/{lon}`），不需要先转和风天气自己的 `LocationID`，所以直接复用 `map_tool.geocode()` 拿坐标，少一次网络调用；`get_hourly_forecast()`（`/weather/v1/hourly/{lat}/{lon}`）同理，且额外传了 `localTime=true`，返回的 `time` 字段直接是当地时间字符串（不带时区偏移），不用再手动转时区
- 和风天气新账号是"一账号一专属域名"：**2023 年后注册的 key 用公共的 `devapi.qweather.com / geoapi.qweather.com` 会直接 404，必须去控制台"项目管理"页面找到这个 key 专属的 API Host（形如 `xxxxxxxxxx.re.qweatherapi.com`），配进 `.env` 的 `QWEATHER_API_HOST`（见"环境准备"一节）——这是接入过程里踩的第一个坑，实测过公共域名连 key 是否有效都判断不出来，报错也不明显（直接 404 空 body，不是 JSON 格式的错误信息）
- 2026-09-11 用真实预警数据验证过：查询当时有台风影响的海南（海口）能查到 2 条真实的雷电/大风黄色预警，字段解析正常；查没有预警的城市（香港/澳门）返回空列表，也验证过降级路径
- 2026-09-12 `get_hourly_forecast()` 真实数据验证过：`nearby_planner.build_plan()` 端到端跑通，拿到真实的澳门逐小时气温/降雨概率，`nearby_sources.weather()` 从这批数据里算出的时段摘要字段跟前端 `nearby.js` 的渲染契约对得上

hotel_tool.py -- 通用酒店工具（道旅 RollingGo）

职责：真实酒店搜索 + 房型报价，给 `ota_hotel_agent.py` 用，2026-09-13 接入。跟 `map_tool.py / weather_tool.py` 一样是通用基础设施。

函数	作用
<code>search_hotels(place, place_type, origin_query, check_in_date=None, stay_nights=1, star_min=None, size=5)</code>	搜酒店候选，返回列表，每条带 <code>hotel_id / name / address / star_rating / price_per_night / lat / lng / image_url / booking_url / amenities</code>
<code>get_hotel_detail(hotel_id, check_in_date, check_out_date, adult_count=2, room_count=1)</code>	查某个酒店的具体房型报价，返回房型列表，带 <code>room_name / price_per_night / bed_type / meal / cancelable / cancel_policy</code>

- 数据源：**道旅（DIDA Travel，官方宣传"全球第三大酒旅 B2B 公司"）的 RollingGo 免费开放接口，200 万+真实酒店库存，个人/企业开发者均可免费申请、目前阶段无调用上限（官方说明是"目前阶段"，不保证长期不变）。去 `https://travelportal-partner-center.dida.com/register?lang=zh` 注册，0 等待即时拿到 key。**这一步是用户自己注册的，不是我能代办的事**（涉及个人账号信息）。
- 协议是 MCP，不是普通 REST：**服务本质是个 Model Context Protocol server（`https://mcp.rollinggo.cn/mcp`），但直接用 `requests` 发 JSON-RPC 2.0 请求过去就行，不用接一整套 MCP 客户端框架——`method` 固定 `"tools/call"`，`params.name` 是工具名，`params.arguments` 是工具参数。**真正的业务数据包了两层：**`response["result"][0]["content"]` 是一段

- JSON 字符串，要再 `json.loads()` 一次才能拿到真正的酒店数据（第一次接的时候没注意到这层，拿到的一直是“文本”外壳，调试了一下才发现）。请求头必须带 `Accept: application/json, text/event-stream`，缺了服务端直接 400。
- **search_hotels() 返回的 price_per_night 是估算值**：源接口 `searchHotels` 只返回整个入住期间的总价（`lowestPrice`），本模块按 `stay_nights` 算术平均换算成“每晚”，不是真的按天查一次；想要某个具体房型精确到每晚多少钱、取消政策，要再调一次 `get_hotel_detail()`（目前 `ota_hotel_agent.py` 没有自动调这一步，只在候选列表阶段用估算价）。
 - **只做只读查询，不碰下单/支付**：官方还有一个走 OAuth 登录 + 人工二次确认的命令行工具（`rgh` 系列，能真实下单付款），调研这个数据源的时候顺带发现的，明确决定不接进这种自动跑的后端 Agent 流程——那类操作必须是用户本人在自己的终端里主动触发，不适合塞进 `orchestrate()` 这种自动化链路。
 - 2026-09-13 真实数据验证过：搜“大三巴附近四星酒店”拿到 5 家真实澳门酒店（假日酒店/华都酒店等），真实价格/地址/坐标/图片/设施；`getHotelDetail` 查到真实房型（假日高级房/高级双床房等）、真实每晚价格、真实取消政策。

agents/orchestrator_agent.py -- 编排 Agent

职责：星型架构的中枢。接收用户消息 → 意图识别 → 决定调用哪几个子 Agent/workflow → 汇总结果 → 生成回复。对应 [agent-interfaces.md 第三节](#) 的节点拓扑，只是从 Dify 可视化节点换成了 Python 函数。模型档位用 `MODEL_FULL`（全系统推理最重）。

2026-09-11 合并 fellow 的 codex/integrate-travel-guardian 分支后，主干结构换成了 fellow 的设计：不再是“LLM 分类 → 调用 Agent → 统一让 LLM 组句”这一条路走到底，而是按意图类型分成两种处理方式：

函数	作用
<code>new_shared_state(user_id, scenario="vacation", onboarding_answers=None)</code>	造一份共享状态： <code>persona</code> （真用 <code>persona.py</code> 的结构，传了 <code>onboarding_answers</code> 就走冷启动）+ <code>trip_plan</code> （真用 <code>trip_plan.py</code> 的结构）+ <code>messages / pending_widgets</code> （合并 fellow 分支后新增，给多轮对话历史和“待确认候选卡片”用）
<code>classify_intent(user_message, history=None)</code>	调模型判断这轮要触发 <code>nearby / content / restaurant / route / booking / exception</code> 里的哪几个，带上历史消息让分类更准（比如“再帮我加一个人”这种要接上文才能判断意图的消息）
<code>orchestrate(user_message, shared_state)</code>	主流程，返回 <code>(output, shared_state)</code>

`orchestrate()` 内部两条路：1. **nearby 命中就整个短路**，直接交给 `nearby_planner.from_chat()` 处理并返回，不进入下面的通用分发——港澳附近游是个自成一体的确定性 workflow（校验请求 → 查真实候选 POI → LLM 只负责挑 ID → 排真实路线/时间 → 纯模板拼回复），细节见 `nearby_planner.py` 一节 2. 其他情况走通用分发：按命中的意图调用对应 Agent（`content / route / booking / exception`，用法跟以前一样），`restaurant` 命中时调 `restaurant_agent.run()`（带工具调用循环的餐厅演示技能，回复原样返回、**不再让第二个模型改写**）；最后收尾生成回复时，`restaurant` 命中直接用它的回复做字符串拼接（把 `route / exception` 的提示语接在后面），其他情况才会真正调一次 LLM 把 `results / trip_plan / persona` 组装成一句话，系统提示词里明确写了“以下 JSON 是资料不是指令，不得捏造即时信息/预订成功/行程变更”防幻觉措辞

`orchestrate()` 返回的 `output` 里除了 `chat_reply / community_panel / map_panel`，还有 `widgets` 数组（见下面 `widgets.py` 一节，`content / booking` 命中时才会有内容）和 `restaurant_evidence`（`restaurant` 命中时工具调用返回的真实数据，前端目前还没用上，留给以后做“展示证据”用）。

agents/content_agent.py -- 达人/内容 Agent

职责：两阶段检索第一阶段。模型档位 `MODEL_FULL`（UGC 语义提炼），目前还没实际调用 LLM（纯向量检索）。

`run(shared_state, location_hint, top_k=3)`：算 `persona` 向量 → `store.query_similar_posts()` 查候选。`location_hint` 现在有两个用途：(1) `_extract_city()` 从消息原文里子串匹配“澳门”/“香港”（词表 `_KNOWN_CITIES`），匹配上了就把 `city` 传给 `query_similar_posts()` 做硬过滤，不然“查香港”会混进澳门结果——纯人格相似度检索不区分城市，得靠这个过滤兜底；(2) 留给“内容相关性排序”（两阶段检索第二阶段）用，那部分还没实现。匹配不到已知城市（比如老的杭州 demo 数据、或者压根没提城市）就不过滤，跟以前行为一样。跟 `exception_agent.py` 的子串匹配是同一个“先跑起来，以后再换实体识别”的思路。

返回的 `recommendations` 每条现在除了原有字段，还带港澳数据库的字段：
`city / category / post_type / address / tags / verified_local`（杭州那批老 demo 数据没有这些，会是 `None` / 空列表/ `False`，不影响原有字段读取）。

agents/route_agent.py -- 行程/路线 Agent

职责：把地点写进 `trip_plan` 某一天的行程节点。模型档位 `MODEL_LIGHT`，目前还没接 LLM，纯结构化写入。

`run(shared_state, places, time_budget=None, city=None)`：所有节点目前都写进同一个占位日期（`_PLACEHOLDER_DAY = "day-1"`）。`arrival_transport` 已经接了 `map_tool.py`，是相邻两站之间真实算出来的交通方式/耗时/距离（超过 2km 自动从步行切驾车）；`arrival_time / end_time` 还是占位文字“待定”（原因见 `map_tool.py` 那节）。

agents/ota_hotel_agent.py -- OTA/酒店 Agent

职责：查库存/比价。模型档位 `MODEL_LIGHT`。**酒店这半边 2026-09-13 起接了真实数据**（`hotel_tool.py`，道旅 RollingGo），**机票这半边还是纯占位假数据**——机票比价没有免费的个人开发者可用来源（携程/去哪儿这类都是企业商家合作性质，不对个人开发者开放，调研过没有绕开的办法）。字段补全到“用户选中后能直接落地”的程度：机票候选带 `flight_no / from_ / to / depart_time / arrive_time / status`，酒店候选带 `address / check_in / check_out`（`check_in / check_out` 会用 `date_range` 参数填，格式“开始日期~结束日期”，没传就默认“明天起 2 晚”）。这样 `POST /widget-response`（见 `server.py` 一节）收到用户选中的候选后，能直接拿这些字段调 `trip_plan.add_flight() / add_hotel()`，不用现造字段。

`run(shared_state, location, date_range=None, category=None, user_message=None)`：`location` 没传就从 `shared_state["city"]` 兜底（再没有就是 demo 默认“澳门”），`user_message` 传了会原样喂给 `hotel_tool.search_hotels()` 当查询意图描述（比拼关键词更准，`orchestrator_agent.py` 的 `booking` 分会把这轮用户原话传进来）；真查失败（key 没配/网络问题）优雅降级成空酒店列表 + `hotel_query_error` 字段，不会抛异常炸穿 `orchestrate()`。

`orchestrate()` 里 `booking` 意图命中时会把这里返回的 `candidates` 分别喂给 `widgets.build_flight_picker_widget() / widgets.build_hotel_picker_widget()`（按 `candidates` 里的 `provider_type` 字段分流，见下面 `widgets.py` 一节），机票假数据、酒店真数据混在同一个候选列表里，两个 widget 都能出内容。

已知缺口（2026-09-13 真实端到端测试时发现）：`orchestrate()` 目前没有从用户消息里提取日期，`date_range` 从来没被传给 `run()`，酒店查询永远用默认的“明天起 2 晚”。真实数据接上之后这个问题第一次变得可见——之前反正是假数据，日期对不对不影响观感；测试时间“9月20-22号”的酒店，LLM 最后收尾造句时诚实地发现拿到的是默认日期的价格，主动跟用户说明了“这不是你问的那两天的价格”（防幻觉提示词生效的证据，没有硬装作数据对上），但功能上确实没做到“按用户说的日期查”。修法待定：可以简单加一段日期正则提取，也可以扩展 `classify_intent` 顺便做实体抽取，需要先决定思路。

agents/exception_agent.py -- 异常应变 Agent

职责：提案制（2026-09-11 合并 fellow 的 `codex/integrate-travel-guardian` 分支后定型）——`run() / check_weather()` 两条评估路径都只评估、不执行，`requires_confirmation` 恒为 `True`，谁都不直接碰 `trip_plan`；`apply_adjustment()`（2026-09-14 新增）才是“用户确认后真正执行”的那一步。模型档位架构文档里写的是“中等模型”，目前只有 `MODEL_FULL / MODEL_LIGHT` 两档，先待定。

函数	作用
<code>run(shared_state, event_type, event_detail=None)</code>	按地点名字匹配 <code>trip_plan</code> 里的行程节点（纯子串匹配， <code>event_detail</code> 是用户整句话），命中就列进 <code>affected_locations</code> ， 不删 。 <code>event_verified</code> 恒为 <code>False</code> ——纯粹是“消息里提到了这个地名”，没有验证过是不是真的发生了
<code>check_weather(shared_state, city)</code>	真查 <code>weather_tool.py</code> （和风天气灾害预警），不依赖用户有没有主动提到天气——只要能确定城市就查真实数据。 <code>event_verified</code> 恒为 <code>True</code> ：跟 <code>run()</code> 的关键区别是这条数据来自真实 API，不是子串猜的，但一样不自动改行程

函数	作用
<code>apply_adjustment(shared_state, place)</code>	真删 ：按地点名字（精确匹配，通常就是前两个函数提案里 <code>affected_locations</code> 的某一项）从 <code>trip_plan</code> 里删掉对应节点，可能跨天/命中多个。返回 <code>{"cancelled": [...], "count": N}</code> ， <code>count=0</code> 不算错误，可能是提案之后行程又被改过

`run()` / `check_weather()` 返回同一套评估契约：
`has_warning` / `needs_replan` / `requires_confirmation` / `event_verified` / `suggested_adjustment`（`run()` 版本另外还有 `affected_locations`）。`apply_adjustment()` 是单独一套“执行结果”契约，不跟评估契约混在一起。

`apply_adjustment()` 底层调的是 `trip_plan.py` 新增的两个函数：`cancel_stop_by_id(trip_plan, node_id)`（按 `node_id` 精确删一个）和 `cancel_stops_by_place(trip_plan, place)`（按地点名字删，可能命中多个）——这两个是对 `trip_plan.remove_stop(day_plan, node_id)` 的封装，调用方不用自己先找是哪一天，直接传整个 `trip_plan` 和 `node_id`/地点名字就行。

这个函数目前没有被 `orchestrate()` 调用——“怎么判断用户确认了、什么时候该调 `apply_adjustment()`”这个触发逻辑还没接（见文末“待接事项”），2026-09-14 这次只是先把“执行”这个接口本身准备好，方便编排 Agent 以后调用。

`check_weather()` 的行为：
- 有真实预警，`needs_replan` 会不会标 `True` 取决于预警等级（`severity`）够不够到 `severe` / `extreme`（常量 `_NEEDS_REPLAN_SEVERITY`）——**但不管等级多高都不会自动改 `trip_plan`**，这只是给编排 Agent 一个“要
不要用更紧迫的语气跟用户说”的信号，不是执行开关（2026-09-11 之前的版本在 `severe` / `extreme` 时会自动清空当天行程，合并
fellow 分支时改掉了，统一成提案制）
- 没有预警返回 `has_warning: False`，是正常情况；`weather_tool` 查询本身失败（key 没配/地名查不到/网络问题）不会抛异常炸穿调用方，优雅降级返回带 `error` 字段的结果——跟 `route_agent.py` 处理 `map_tool` 查询
失败是同一个思路
- `orchestrator_agent.py` 里 `exception` 意图命中时会额外跑一次：从用户消息里提取城市（`_KNOWN_CITIES =`
（“澳门”，“香港”），跟 `content_agent.extract_city` 同一套子串匹配思路，独立一份没有互相 `import`，两个模块本来就不该耦
合），提取到了才调用 `check_weather()`，结果挂在 `results["exception"]["weather_check"]` 里，最后收尾的 LLM（或
`restaurant` 命中时的字符串拼接）会把这个提案说给用户听 - **不写 `trip_plan.weather_alerts`**：提案制意味着“记录下来”本身也算
一种执行，所以这版故意不调 `trip_plan.add_weather_alert()` ——`GET /trip` 暂时看不到这些真实预警，等“确认后执行”那个接口
做出来了再一并把写入这一步补上

widgets.py -- 展示插件（右侧聊天框富交互组件）

职责：右侧聊天框里除了纯文字回复之外的富交互小组件（帖子列表、景点勾选等），跟 Agent/编排逻辑解耦——组件只负责“从真实候选集里挑/怎么展示”，不重新跑检索或编内容。

设计原则：
- 候选集永远来自真实代码算出来的数据（目前是 `content_agent.run()` 的 `recommendations`，已经是 `store.query_similar_posts()` 按人格相似度排过序的真实结果），不允许凭空捏造。
- 要用 LLM 做“选择/排序”时，LLM 只能从候选集里选 ID，不允许自己编内容；`_llm_select()` 会校验 LLM 返回的 ID 是否都在候选集里，筛掉任何编造的 ID（防幻觉），LLM 调用/解析失败时优雅降级成“按候选集原有顺序截取前 N 个”，不影响整体流程。
- 每个 widget 统一格式 `{"widget": "<type>", "data": {...}}`，前端按 `widget` 类型分发渲染，渲染逻辑完全不用管数据是怎么算出来的。
- 模型档位：`_llm_select()` 内部用 `llm_tool.MODEL_LIGHT`（选择/排序是结构化任务，不需要强模型）。

函数	作用
<code>build_post_list_widget(candidates, top_k=3, use_llm_rerank=False)</code>	帖子展示插件：默认直接截取候选集前 <code>top_k</code> 个（候选集已经是真实排序过的，不用额外调 LLM）；传 <code>use_llm_rerank=True</code> 走 LLM 二次筛选
<code>build_attraction_picker_widget(candidates, max_select=3, pool_size=6, use_llm_rerank=True)</code>	景点选择插件：从候选里选出 <code>pool_size</code> 个放进“可选池”返回给前端，用户从池子里最多勾 <code>max_select</code> 个；默认走 LLM 精选候选池（LLM 只能选真实 ID）

函数	作用
<code>build_flight_picker_widget(candidates, max_select=1, pool_size=3, use_llm_rerank=True)</code>	机票选择插件: 跟 <code>build_attraction_picker_widget</code> 同一套"可选池 + 前端勾选 + 结果回传"契约, 只是机票通常只订一个, 默认 <code>max_select=1</code> ; 从 <code>candidates</code> 里只挑 <code>provider_type == "flight"</code> 的, 走 LLM 精选出 <code>pool_size</code> 个放进候选池; 筛出来是空的就返回 <code>None</code> (调用方按 <code>None</code> 跳过, 不展示空插件)
<code>build_hotel_picker_widget(candidates, max_select=1, pool_size=3, use_llm_rerank=True)</code>	酒店选择插件: 跟 <code>build_flight_picker_widget</code> 同一个模式, 只挑 <code>provider_type == "hotel"</code> 的; 同样可能返回 <code>None</code>

依赖: - `content_agent.py` 返回的 `recommendations` 每条都带 `post_id` 字段 (`store.query_similar_posts()` 本来就返回这个字段, 之前 `content_agent.py` 组装返回值时漏传了, 现已补上), `build_post_list_widget / build_attraction_picker_widget` 靠这个字段做 ID 校验。 - `build_flight_picker_widget / build_hotel_picker_widget` 的候选 (`ota_hotel_agent.run()` 的 `candidates`) 没有独立 id 字段, 改用 `name` 字段当 `_llm_select` 的校验 key——跟 `post_id` 是同样的作用, 只是换了个字段名。

四个 widget 分两种交互模式: `post_list` 是纯展示 (截取直接列出来); `attraction_picker / flight_picker / hotel_picker` 是"可选池 + 前端勾选 + 结果回传"——`options + max_select` 是统一契约, 机票/酒店默认 `max_select=1` (通常只订一个), 景点默认 `max_select=3` (可以多选几个)。

现状: 四个 widget 都已经真正接进 `web/index.html` 渲染, `POST /widget-response` 接口也做完了 (见 `server.py` 一节) ——右侧聊天框里能看到真实的卡片/勾选交互, 勾完点"确认选择"会真的把机票/酒店写进 `trip_plan`、把景点排进路线, 不再是"传了但没人用"的状态。

待接事项: - 讨论过的其他 widget 想法还没做: 反馈评分插件、异常变更确认插件、人格问卷引导插件

schedule_widgets.py -- 左侧日程面板展示组件

职责: 服务对象是左侧实时日程面板 (`server.py` 的 `GET /trip`, `web/index.html` 目前的文字列表), 跟右侧聊天框的 `widgets.py` 是同一个"数据/展示分离"思路——这里只算"给前端画图/排版用的数据", 不产出任何 HTML/画布逻辑, 真正怎么画留给前端。模型档位: 不需要 LLM, 纯确定性计算 (按天/类型分配颜色 + 查真实地理数据)。三个 widget:

函数	作用
<code>build_trip_map_widget(trip_plan_obj, city=None)</code>	地图组件: 把每天的景点/餐饮节点标进地图 (同一天同一个颜色, 颜色按"第几天"从固定调色板分配, 超过 8 天循环复用), 并把当天节点依次连成一条 真实路线 (查真实路况, 不是直线连线, 颜色跟当天 <code>markers</code> 一致); 机票起降机场、酒店地址也各标一个点, 用固定颜色 (不参与按天上色, 因为它们不属于"某一天")。多城市行程会按地理距离自动 分帧 (见下), 不再是所有点挤在同一张地图里
<code>build_day_timeline_widget(trip_plan_obj)</code>	每日行程时间线: 把每天的节点按时间顺序整理成竖排列表, 按天分模块, 模块颜色跟 <code>build_trip_map_widget</code> 里同一天的颜色对齐, 方便地图和列表对照着看
<code>build_booking_panel_widget(trip_plan_obj)</code>	机票/酒店面板: 原样呈现 <code>trip_plan</code> 里已经落地确认的 <code>flights / hotels</code> 记录——注意这跟 <code>widgets.py</code> 的 <code>build_flight_picker_widget / build_hotel_picker_widget</code> 不是一回事, 那两个是"聊天框里给用户挑的候选推荐", 这里是"已经确认、要显示在左侧行程里的记录"

`build_trip_map_widget` 返回结构 (分帧版, 2026-09-11 起替换了原来的扁平 `markers / routes` 结构):

```
{
  "widget": "trip_map",
  "data": {
    "frames": [
      {
        "frame_id": "frame-1",
        "center": { "lng": ..., "lat": ... },
        "markers": [
          {
            "day": ...,
            "color": ...,
            "place": ...,
            "node_id": ...,
            "type": "attraction|meal|airport|hotel",
            "routes": [
              {
                "day": ...,
                "color": ...,
                "coordinates": [
                  [
                    "lng", "lat"
                  ],
                  ...
                ]
              },
              ...
            ]
          },
          ...
        ],
        "unclustered_airports": [
          {
            "day": ...,
            "color": ...,
            "place": ...,
            "node_id": ...,
            "type": "airport",
            "role": ...,
            "flight_no": ...,
            "status": ...,
            "lng": ...,
            "lat": ...
          },
          ...
        ]
      },
      ...
    ]
  }
}
```

```

"days_legend": [{"date":, "color":}],
"geocode_failures": [{"place":, "reason":}],
"route_failures": [{"day":, "from":, "to":, "reason":}],
}}

```

- **为什么要分帧**：早期版本是所有 marker 挤进同一张地图，跨城市行程（比如“杭州玩三天+机票飞北京”）一旦机票起降机场跟行程主城市离得远，地图会被迫缩到能同时装下两座城市比例尺，近处的景点全部挤成一个点，什么都看不清。改成按地理距离把 marker 分组，每组（帧）各自算自己的地图范围/比例尺，前端各画一个独立的 AMap.Map 实例，近处的点就能放大到看得清的程度
- **分帧算法**（schedule_widgets.py 里的 _cluster_by_distance + _haversine_m）：对当天的景点/餐饮/酒店 marker 用 haversine 距离做贪心单链聚类，阈值 _FRAME_CLUSTER_THRESHOLD_M = 50_000（50km）——两个点只要有一条 ≤50km 的链路就会被分进同一帧，超过阈值的自动分到不同帧
- **机票起降机场不参与聚类本身**，而是聚类结束、每帧的中心点算出来之后，拿机场坐标跟每一帧的中心点比距离，落在 _AIRPORT_ATTACH_THRESHOLD_M = 80_000（80km）以内就并入最近的那一帧（进 frames[i].markers），否则单独放进 unclustered_airports（不画在任何一帧里，避免为了容纳一个远处的机场把某一帧的比例尺又拉爆）——这是用户明确要求的规则：“起点和终点不算，直接忽略就行”
- 景点/餐饮 marker 的 day / color 按天分配；机票（type: "airport"，额外带 role: "depart"|"arrive"、flight_no、status）和酒店（type: "hotel"，额外带 name / check_in / check_out）的 day 固定是 None，颜色分别固定为深灰蓝/棕色（_AIRPORT_COLOR / _HOTEL_COLOR），不占用按天调色板
- 每帧内 routes 的坐标是把 map_tool.route_between() 返回的多段 polyline 展平成一条连续的 [lng, lat] 序列，直接给前端画线；一条路线按起点锚点归到最近的帧里；查询失败的那一段跳过并记进 route_failures，不影响其他天/其他段
- **字段约定**：flights 的 from / to、hotels 的 address 建议存能被地理编码识别的地名/地址（比如“杭州萧山国际机场”），不建议只存三字码（比如 "HGH"）——高德地理编码认不出机场三字码，会直接进 geocode_failures

现状：三个 widget 已经接进 server.py 的 GET /trip，web/index.html 用高德 JS API 真的把地图画出来了（见 server.py + web/index.html 一节）；已经用真实 AMAP_KEY 端到端验证过，标点/连线/机票酒店坐标都是真实经纬度。

现状/限制：- 跟 route_agent.py 一样依赖 AMAP_KEY（后端查经纬度/路线用的那个，注意不是前端画图用的 AMAP_JS_KEY），没配置时 markers / routes 会是空的、*_failures 里全是查询失败记录（不会导致整体报错，自测+真实端到端联调都验证过这个降级路径）- route_agent.py 里已经算过一次相邻站点的交通方式/耗时，这里为了拿到完整 polyline 又独立查了一次路线（避免“展示层”反过来依赖某个 Agent 的内部计算结果），会有一点重复的高德 API 调用，demo/比赛规模的免费额度足够用，不是问题 - POST /widget-response 确认机票/酒店后会真的调 trip_plan.add_flight() / add_hotel()，build_booking_panel_widget / 机票酒店 marker 现在跑起来是有真实数据的（端到端联调过：选中机票+酒店 → /trip 能看到确认记录）- **机票起降机场地理编码不传 city**：build_trip_map_widget(trip_plan_obj, city=...) 的 city 参数只用于景点/餐饮/酒店这些“行程主城市内”的地点，机场坐标查询故意不传 city——接真实 key 测试时踩到过一坑：查“上海虹桥国际机场”时如果传了 city="杭州"，高德会把它强行匹配成杭州萧山机场附近的结果（两个机场坐标几乎重合），因为出发/到达机场天然可能分属两个不同城市，用行程主城市限定反而会查错

main.py -- 入口脚本

职责：纯胶水代码，不包含任何 Agent 逻辑。from agents.orchestrator_agent import new_shared_state, orchestrate 之后直接 re-export，server.py 还是 import main 调这两个函数，完全不用改。__main__ 里是完整的端到端 demo（推荐 → 排路线 → 异常应变 → 打印最终行程）。

store.py -- 达人社区内容 + 历史记录

职责：达人社区的图文帖子存储 + 检索（两阶段检索第一阶段：按人格向量相似度筛选），以及行程反馈历史记录。

存储技术：Chroma 向量库（orchestrator/data/chroma/，本地文件持久化）存帖子；JSON Lines 文件（orchestrator/data/history.jsonl）存历史记录；图片本地落盘（orchestrator/data/images/<post_id>/），帖子里只存相对路径。

函数	作用
<code>save_image(post_id, filename, image_bytes)</code>	写一张图，返回存进 <code>content["images"]</code> 的相对路径
<code>resolve_image_path(相对路径)</code>	相对路径转本地绝对路径，读图用
<code>add_post(post_id, persona_vector, content)</code>	存一条帖子。 <code>content</code> 字段见下表
<code>query_similar_posts(persona_vector, top_k=5, city=None)</code>	两阶段检索第一阶段：按人格向量相似度找候选帖子，返回时带 <code>similarity_score</code> ； <code>verified_local=True</code> 的帖子会被优先加权（见下面说明）；传了 <code>city</code> 会在 Chroma 查询阶段就用 <code>where={"city": city}</code> 精确过滤，不是"先按相似度截 <code>top_k</code> 再筛掉不match的"（那样可能筛到结果不够甚至是空的）
<code>delete_post(post_id)</code>	删帖
<code>log_history(user_id, trip_id, record)</code>	追加一条行程反馈记录
<code>get_history(user_id=None)</code>	读历史记录，不传 <code>user_id</code> 读全部

`content` 字段（`add_post` 的第三个参数）：

字段	说明
<code>place / time_slot / avg_cost / rating / avoid_tips / verified_trip</code>	结构化字段，对齐 <code>agent-interfaces.md</code> 达人 Agent 输出契约
<code>caption</code>	帖子正文文字，可选
<code>images</code>	图片相对路径列表，可选，由 <code>save_image()</code> 生成
<code>tags</code>	标签列表，可选，跟 <code>images</code> 一样序列化存（ <code>tags_json</code> ），查询时还原回列表
<code>city / category / post_type / address / verified_local</code>	港澳达人数据库用的字段（见 <code>import_hk_macau_data.py</code> 一节）， <code>verified_local=True</code> 表示本地人认证来源

"本地人认证"优先加权是怎么做的： `query_similar_posts()` 先按 `top_k * 3` 多捞一些候选（不是只捞 `top_k` 个），把 `verified_local=True` 的帖子相似度乘一个固定系数 `_VERIFIED_LOCAL_BOOST = 1.15`，再按加权后的分数重新排序截到 `top_k`。这个"先多捞再重排"是必须的——如果直接对 Chroma 已经按原始距离截好的 `top_k` 结果加权，加权只能在这几条里面换个顺序，换不进被漏掉的本地人帖子，加权就没意义了。

待定/限制： - 内容相关性排序（两阶段检索第二阶段）没实现，留给调用方在 `query_similar_posts()` 的候选集里自己排 - 图片存本地文件系统，部署到 ModelScope Studio 时要换成对象存储（OSS/CDN），`images` 字段到时候存 URL 而不是本地路径，上层调用方式不变 - `_VERIFIED_LOCAL_BOOST` 系数（1.15）是拍的，没有做过效果对比调优，觉得加权效果不明显/太强可以直接改这个常量

standardize_hk_macau_data.py + import_hk_macau_data.py -- 港澳达人数据库

背景： 组员手动整理了一份港澳（澳门/香港）达人推荐/避坑数据，存在仓库根目录的 `数据库原始.xlsx` 里，来源分两种： - **本地人认证**（Excel 里的"本地人澳门"/"本地人香港" sheet）： 饮食/景点/Tips 分类推荐 + 单独一张避坑表 - **小红书**（"小红书澳门"/"小红书香港" sheet）： 褒贬都有的普通帖子，不算本地人认证来源

这两种来源对应 `store.py` 里新加的 `verified_local` 字段——本地人认证的帖子在检索时会被优先加权（见 `store.py` 一节），小红书来源不加权。这个字段是原始数据自带的真实来源标记（组员整理的时候已经分好表了），不是靠算法从内容"判断"出来的。

两步流程（分开是为了标准化出错时不用重新灌一次向量库）：

脚本	作用
<code>standardize_hk_macau_data.py</code>	读 数据库原始.xlsx，标准化字段 + 编号，写出 <code>data_sources/hk_macau_posts.json</code> 。不碰 Chroma
<code>import_hk_macau_data.py</code>	读标准化 JSON，用 <code>persona.infer_post_persona_vector()</code> 给每条帖子算一个人格向量，调 <code>store.add_post()</code> 灌进 Chroma

编号规则（城市前缀 + 类型字母 + 三位数字，类型内连续编号，不区分本地人/小红书来源）：

	澳门·推荐	澳门·避坑	香港·推荐	香港·避坑
编号格式	MF/MS/MT + 三位数字	MN + 三位数字	HF/HS/HT + 三位数字	HN + 三位数字
示例	MF001, MS001, MT001	MN001, MN002	HF001, HS001, HT001	HN001, HN002
说明	饮食/景点/Tips，正常分类	全部统一放这里，不分饮食/景点	同左	同左

解读：**M**=Macau，**H**=Hong Kong（城市前缀）；**F**=Food，**S**=Sight，**T**=Tips（推荐分类）；**N**=No（避坑专用，统一放一起不细分）。

原始表格解析上的坑：Excel 里不同分类小节的列不是对齐的（比如“本地人香港”的 Tips 小节没有单独的“名称”列，内容直接写在名称那一格；“本地人香港”饮食小节比“本地人澳门”多一列“推荐人”），`standardize_hk_macau_data.py` 按每个小节自己的表头动态取列，不能假设固定列位置——这个坑是解析的时候踩出来的，写死列位置会导致数据错位。

当前数据量（229 条，跑 `python standardize_hk_macau_data.py` 会打印这个分布）：

前缀	条数	前缀	条数
MF	50（本地人30+小红书20）	HF	19（本地人9+小红书10）
MS	50	HS	20
MT	50	HT	20
MN	10	HN	10

已知数据缺口：香港饮食类本地人认证只有 9 条，不是目标的 10 条——原始表格里就是这样，组员应该是漏填了一条，不是解析脚本的 bug，不影响其他数据使用。

怎么重新跑：

```
python orchestrator/standardize_hk_macau_data.py # 改了 Excel 或标准化逻辑之后跑
python orchestrator/import_hk_macau_data.py      # 改了人格反推逻辑，或者清空过 orchestrator/data/chroma 之后跑
```

`import_hk_macau_data.py` 用的是 `upsert`（`store.add_post()` 内部调的），同一个 id 重复导入会覆盖不会重复，可以放心重跑。

待接事项：- 避坑类帖子（MN / HN）反推出来的人格向量是中性默认值（见 `persona.py` 一节的已知局限），检索排名上不占优势也不吃亏，更合理的做法（不管人格匹配度、只要地点/类型对上就该出现）属于两阶段检索第二阶段，还没实现 - `verified_local` 优先加权的系数（1.15）没有做过 A/B 效果对比，是拍的经验值 - `content_agent.py` 的两阶段检索第二阶段接上之后，`category`（饮食/景点/Tips）字段可以用来做更精细的“用户问吃的就优先出饮食类”这种过滤，现在还没用上

trip_plan.py -- 旅游计划（行程/机票/酒店/天气异常）

职责：一次完整旅行的所有结构化信息，由多个 Agent 共同维护，最终汇总用于呈现。

顶层结构：

```
trip_plan = {
    "trip_id": "...",
    "flights": [...],          # OTA/酒店 Agent 维护
    "hotels": [...],          # OTA/酒店 Agent 维护
    "weather_alerts": [...], # 异常应变 Agent 维护
    "days": {日期: day_plan},# 行程 Agent 生成, 异常应变 Agent 增删改
}
```

景点通勤+餐饮的核心设计: 每天一个"链表", 但不是指针式链表, 而是**节点字典 + id 引用** (day_plan["nodes"] 是 {node_id: node}, 每个节点有 next_id 指向下一个节点的 id, day_plan["head_id"] 是链表头)。这样设计是因为: - 异常应变 Agent 改行程时只需要动被影响的那一个节点、接一下前后 next_id, 不用重发整天的数组 - 本质是个普通 dict, 直接能存 JSON/传 HTTP, 不需要额外的链表↔数组转换逻辑

节点字段: type ("attraction" | "meal")、place、arrival_transport、arrival_time、end_time、next_id; meal 类型节点多一个 meal_type (早餐/午餐/晚餐/夜宵)。

函数	作用
new_trip_plan(trip_id)	造一个空旅游计划
add_flight / add_hotel / add_weather_alert	往对应列表追加一条记录
get_or_create_day(trip_plan, date)	拿到某天的 day_plan, 不存在就新建
add_stop(day_plan, node_id, type_, place, arrival_transport, arrival_time, end_time, after_id=None, **extra)	插入一个节点, after_id=None 插到最前面, 否则插到该节点后面
remove_stop(day_plan, node_id)	删除节点, 自动重连前后节点。低层原语, 调用方要先自己找到 day_plan
cancel_stop_by_id(trip_plan, node_id)	2026-09-14 新增: 按 node_id 删节点, 不用先自己找是哪一天——遍历 trip_plan["days"] 定位到再删, 返回被删节点内容 (带 date / node_id) 或 None
cancel_stops_by_place(trip_plan, place)	2026-09-14 新增: 按地点名字 (精确匹配) 删节点, 可能跨天/命中多个, 返回被删节点列表。这两个是 remove_stop() 的高层封装, 给 exception_agent.apply_adjustment() 用, 也是编排 Agent"确认后执行"时应该调的接口, 不建议跳过封装直接调 remove_stop()
patch_stop(day_plan, node_id, **fields)	只改某个节点的部分字段, 不动链表结构
day_stops(day_plan)	把链表还原成有序数组, 渲染/地图面板用
render(trip_plan)	汇总成最终展示结构 (flights / hotels / weather_alerts / days , 每天的 stops 已经是有序数组)

现状: 已经接进 main.py 的 shared_state["trip_plan"], route_agent 会往里面加节点。exception_agent 现在是提案制, 不会自己删节点——真要删得靠编排 Agent 调 exception_agent.apply_adjustment() (内部调 cancel_stops_by_place()), 但这一步还没接进 orchestrate() (见"待接事项")。目前所有节点都写进同一个占位日期 "day-1" (main.py 里的 _PLACEHOLDER_DAY), 还没做真正的多日期规划。

persona.py -- 人格变量

职责: 定义"旅游人格"的具体维度结构, 供达人/行程/OTA Agent 读取, 反馈闭环用来校准。

维度设计 (量化的维度都用了 0~1 连续分数, 不是纯标签) :

维度	归属	类型	说明
taste	stable_traits（跨场景稳定）	固定词表 multi-hot	菜系标签，词表见 _TASTE_VOCAB
novelty_score	stable_traits	float 0~1	0=稳妥大众，1=小众冒险
pace_score	scenario_traits（按场景分开存）	float 0~1	0=完全休闲，1=完全特种兵
budget_score	scenario_traits	float 0~1	0=穷游，1=公务舱/总统套房/租车这种享受型消费；硬过滤字段，不进相似度向量，用 budget_filter_ok() 按数值区间过滤
social_mode	scenario_traits	分类（solo / couple / family / friends）	并列模式非程度光谱，向量里按 one-hot 编码
interest_theme	scenario_traits	固定词表 multi-hot	词表见 _INTEREST_VOCAB

scenario_traits 按"本次旅行模式"（business_trip / vacation / family / solo_adventure ...）分开存，同一个人不同场景下这些值可以完全不同。

函数	作用
new_persona(user_id)	造一份空人格
bootstrap_from_onboarding(user_id, scenario, answers)	冷启动：从一份问卷答案组出初始 persona
set_scenario_traits(persona, scenario, **fields) / get_scenario_traits(persona, scenario)	读写某场景的特质，写入时会校验词表 + clamp 数值到 0~1
update_stable_traits(persona, **fields)	读写跨场景稳定特质，同样有校验
apply_feedback(persona, scenario, dimension, new_value, reason="")	按维度校准，写入 + 留痕（feedback_log）。"该改成什么值"由调用方决定，这里只负责写入
budget_filter_ok(a_score, b_score, max_diff=0.25)	判断两个 budget_score 是否够接近，硬过滤专用，独立于相似度向量
compute_persona_vector(persona, scenario)	拼出向量：[novelty_score, taste multi-hot, pace_score, social_mode one-hot, interest_theme multi-hot]，固定 21 维（词表大小变了维度也会变，加了"葡国菜"后从 20 变 21）
infer_post_persona_vector(scenario, tags=None, avg_cost=None, category=None)	港澳达人数据库导入专用：这些帖子是组员手动整理的真实内容，没有"发帖人自己的问卷答案"，用标签/人均/分类反推一个大致合理的人格向量，见下面单独说明

infer_post_persona_vector() 是怎么反推的（启发式估计，不是精确画像）：- _TAG_TO_TASTE / _TAG_TO_INTEREST 两个关键词映射表，对标签文本做子串匹配，匹配上就计入对应的 taste / interest_theme；匹配不上的标签（比如"三代传承"400年历史"这种描述性但不构成人格维度信号的）直接丢弃，不强行凑 - category（饮食/景点）也会补一条弱信号：饮食→"美食探店"，景点→"人文历史" - avg_cost 线性换算成 budget_score（0~250 元映射到 0~1，250 元封顶），换算公式很粗糙，不是精确定价模型 - 标签里出现"小众/冷门/隐世"这类词 → novelty_score 偏高（0.75）；出现"网红/热门/必打卡" → 偏低（0.3）；都没有就是中性 0.5 - social_mode 故意不猜——单条帖子看不出"适合独行/家庭/朋友"，留空对应 compute_persona_vector() 里那段 one-hot 全 0，这是合法的"未指定"编码 - **已知局限**：避坑（避雷理由）类帖子在标准化数据里没有 tags / avg_cost / category，反推出来的人格向量是中性默认值（novelty=0.5, budget=0.5，无 taste/interest），跟任何用户人格的相似度都不会特别高也不会特别低——这类帖子更适合"不管人格匹配度、只要地点/类型对上就该出现"，属于两阶段检索第二阶段（内容相关性排序）该管的事，那部分还没实现

待定/限制：- compute_persona_vector() 是数值特征拼接，不是真实语义 embedding，以后接真实 embedding 模型可以整体替换掉函数体，调用方（store.py）接口不用变 - 改 _TASTE_VOCAB / _INTEREST_VOCAB 词表会改变向量维度，如果本地 Chroma 已经

有存量数据，改词表前要清掉 `orchestrator/data/chroma` 重新灌数据，不然新旧向量维度对不上（这个坑已经踩过一次，这次加“葡国菜”也重新灌了一次数据） - 从一条用户反馈文本判断“该把哪个维度调成什么值”的算法没实现，留给编排/达人 Agent 决定

server.py + web/index.html -- 呈现层（网页）

职责：把 `main.py` 的编排 Agent、`widgets.py`、`schedule_widgets.py` 包成 HTTP 接口 + 真实渲染，配一个两栏网页——左边实时日程（地图+每日时间线+机票酒店面板），右边聊天框（文字回复+四个插件卡片）。这是目前唯一“从对话到落地预订”整条链路能真的跑通的地方。

运行方式：

```
pip install flask python-dotenv
python orchestrator/server.py
```

然后浏览器打开 `http://127.0.0.1:5000`。地图能不能真的画出来，取决于 `.env` 里三个高德 key 有没有配全（见“环境准备”一节）；没配的话页面照样能跑，只是地图是空的、控制台会有一条警告。

会话隔离（2026-09-11 合并 fellow 分支后新增）：每个浏览器发一个 `travel_session` cookie（32 位十六进制随机值），后端靠这个 cookie 从 `session_store.py`（SQLite）读写各自独立的 `shared_state`，不再是全局共享一份——不同人/不同浏览器打开不会互相覆盖行程和对话记录，但仍然是单进程本地 demo，没有账号登录体系。

接口（都是同一个 Flask app 提供，同源不用处理 CORS）：

接口	作用
GET /	返回 <code>web/index.html</code> ，服务端会把文件里的 <code>__AMAP_JS_KEY__</code> / <code>__AMAP_JS_SECURITY_CODE__</code> 占位符换成 <code>.env</code> 里的真实值再返回（这两个 key 不写进 git 里的 html 文件，跟其他密钥一样只活在 <code>.env</code> ）
GET /session	页面刷新/重新打开时用来恢复对话历史和待处理 widgets（ <code>pending_widgets</code> ），前端 <code>restoreSession()</code> 调这个
GET /trip	返回 <code>trip_plan.render()</code> 的行程 + 额外几个字段 <code>trip_map</code> / <code>day_timeline</code> / <code>booking_panel</code> （ <code>schedule_widgets.py</code> 那三个函数的输出）+ <code>nearby_plan</code> （如果这个 session 生成过港澳附近行程的话）
POST /chat	body 传 <code>{"message": "...}"</code> ，内部调 <code>main.orchestrate()</code> ，返回编排 Agent 的输出（ <code>chat_reply</code> / <code>community_panel</code> / <code>map_panel</code> / <code>widgets</code> / <code>restaurant_evidence</code> ）
POST /widget-response	body 传 <code>{"widget": "attraction_picker" "flight_picker" "hotel_picker", "selected": [...]}</code> ， <code>selected</code> 是前端从对应 widget 的 <code>options</code> 里原样拿到的候选对象。服务端会先校验这个 widget 是不是这个 session 当前真的“待处理”（ <code>pending_widgets</code> ），选中项是不是真的在候选池里、有没有超过 <code>max_select</code> 、有没有重复——都是合并 fellow 分支带来的加固，防止前端被篡改后伪造候选。景点选中后调 <code>route_agent.run()</code> 排进行程；机票/酒店选中后调 <code>trip_plan.add_flight()</code> / <code>add_hotel()</code> 确认预订
POST /nearby-plan / POST /nearby-weather / GET /nearby.js	港澳附近游专用接口：生成行程（调 <code>nearby_planner.build_plan()</code> ）、单独刷新天气（不用重新排一遍行程）、拉取前端脚本。详见 <code>nearby_planner.py</code> 一节

前端逻辑（`web/index.html`，原生 HTML/CSS/JS + 高德地图 JS SDK，没引入前端框架）：

- 发消息 → 调 `/chat` → 显示文字回复 → 把返回的 `widgets` 数组按类型分发渲染（`post_list` 是横滑卡片；`attraction_picker` / `flight_picker` / `hotel_picker` 共用同一套“复选框 + 确认按钮”组件，`max_select` 决定最多能勾几个）→ 再调一次 `/trip` 刷新左侧日程
- 三个“选择型”widget 点“确认选择”之后：把勾中的候选对象原样 `POST /widget-response`，成功后往聊天记录里加一条系统提示（“已确认：xxx”），再刷新一次左侧日程
- 左侧地图不再是页面加载时初始化一次的单个 `AMap.Map`（2026-09-11 起改为分帧渲染，跟后端 `trip_map.data.frames` 的结构对应）：每次 `/trip` 刷新，`renderMap()` 先把上一轮所有帧的 `AMap.Map` 实例逐个 `destroy()`，再按 `frames` 数组动态生成对

应数量的 `.map-frame` 容器（`#map-frames` 下面纵向排列，每帧一个独立的 220px 高地图 div + “第 N 组 · X 个地点” 标签），
`drawFrameMap()` 给每帧各自 `new` 一个 `AMap.Map` 实例、只画这一帧自己的 `markers / routes`，各帧自动按自己的点位算合适的
缩放级别（不用整条行程的极值），互不干扰

- 景点/餐饮用 `AMap.CircleMarker` 按天上色，机票/酒店 marker 固定深灰蓝/棕色（颜色跟 `schedule_widgets.py` 里的常量对应，改了那边记得这边也要改），路线用 `AMap.Polyline` 按天上色，标记点击会弹 `AMap.InfoWindow` 显示地点名（这部分单帧内部的画法逻辑跟分帧之前一样，没变）
- `data.unclustered_airports`（离行程主体太远、没并进任何一帧的机场）渲染成 `#map-frames` 下方一行文字提示（“距离行程较远，未画在地图上：xxx”），不占用地图画布
- 每日行程时间线现在读 `trip.day_timeline`（带颜色）而不是原始的 `trip.days`，模块颜色跟地图上同一天的颜色对得上

现状/限制： - 按浏览器 session 隔离状态（见上面“会话隔离”），但仍是单进程本地 demo，没有账号登录体系/多进程并发处理，仅供本地演示，不要直接这样部署到公网 - 已经端到端联调过完整链路：“推荐+排路线+查机票酒店”发过去 → 右侧出现 `flight_picker / hotel_picker` → 模拟前端 `POST /widget-response` 选中一个航班一个酒店 → 再查 `/trip` 能看到 `flights / hotels` 里多了确认记录（这一步是直接打 API 验证的，没配 `AMAP_JS_KEY` 时浏览器里的地图看不到，但数据链路是通的） - 社区图文面板（三面板设计里“逛社区帖子墙”那块，跟聊天框里的 `post_list` 不是一回事）还没做进网页 - 地图是“重新全量清空再画”，不是增量更新，行程节点很多的时候会有一点点闪烁，demo 规模不明显

待接事项

`main.py / server.py` 已经把所有 Agent + 两套 widgets 串成一条完整链路：跑 `python orchestrator/main.py` 能看到一次“推荐→规划路线→异常应变→重新渲染行程”的端到端流程；跑 `python orchestrator/server.py` 打开网页，能看到“聊天推荐→勾选景点/机票/酒店→确认→左侧地图/时间线/机票酒店面板更新”这条更完整的闭环真的在跑（已端到端联调验证过）。还剩这几处没做，都不阻塞基本演示：

1. `ota_hotel_agent` 还是纯占位假数据~~ 酒店这半边已解决（2026-09-13）：接了 `hotel_tool.py`（道旅 RollingGo，真实酒店库存/价格）；机票还是纯占位假数据，没有真实比价来源可接（调研过携程/去哪儿都是企业商家合作性质，个人开发者拿不到免费 key）。**道旅 RollingGo 的 Partner Center 后台其实有机票产品**（用户 2026-09-13 在自己的后台账号里确认看到过），但当时状态是“已下线维护”，没能拿到 endpoint/文档——等它恢复了优先去接这个，不用再重新调研数据源，直接照 `hotel_tool.py` 的模式（MCP JSON-RPC over HTTP，业务数据包两层）写一个 `flight_tool.py` 大概率能直接抄
2. `route_agent` 目前所有节点都写进同一个占位日期 `_PLACEHOLDER_DAY = "day-1"`，没有真正的多日期规划——这意味着“按天上色”这个视觉设计在真实多日行程里还体现不出来（永远只有一种颜色）；交通方式/耗时已经接了高德地图 API 真实计算，但到达/结束的具体钟点时间还是占位文字“待定”
3. `exception_agent` 用整句用户消息去子串匹配地点名字，很粗糙（比如“西湖”两个字出现在消息里就命中），真实版本应该先做实体识别抽出具体地点
4. 两阶段检索第二阶段（候选集内部按内容相关性排序）还没实现，`content_agent` 里 `location_hint` 参数目前没用上
5. 反馈闭环（用户点评行程 → 校准 persona）还没接：`store.log_history()` 记录和 `persona.apply_feedback()` 校准都写好了，但没人在 `orchestrate()` 里调用它们
6. `map_tool.py` 目前只查“两点之间”，没做“多点最优顺序”规划（比如给定 5 个景点，没有算出最优游览顺序，只是按用户/LLM 给的顺序排）
7. 三个高德 key 已经申请配置好并真实验证过（西湖→灵隐寺路线、机场/酒店坐标都是真实高德数据）
8. 讨论过的其他 widget 想法还没做：反馈评分插件、异常变更确认插件、人格问卷引导插件
9. ~~单会话全局 state~~ 已解决：合并 fellow 分支后改成 `session_store.py`（按浏览器 cookie 隔离，SQLite 存档），仍然是单进程本地 demo，没有登录/账号体系，但至少不同浏览器/不同人打开不会互相覆盖状态了
10. 港澳达人数据库（229 条，见 `standardize_hk_macau_data.py` 一节）已经标准化+导入 Chroma，`verified_local` 优先加权、`city` 硬过滤（`content_agent._extract_city()` 子串匹配）都接上了并真实验证过（问香港只出香港、问澳门只出澳门）；但避坑类帖子的人格向量是中性感默认值、`category`（饮食/景点/Tips）字段还没用来做精细过滤，这两个属于两阶段检索第二阶段（第 4 条）的范畴
11. **~~ exception_agent 提案制的“确认后执行”接口还没做~~ 接口本身已解决（2026-09-14）**：新增 `exception_agent.apply_adjustment(shared_state, place) +`
`trip_plan.cancel_stop_by_id() / cancel_stops_by_place()`，编排 Agent 拿着提案里的地点名字调这个就能真删节点，已用真实数据自测过。**但触发逻辑还没接**：`orchestrate()` 里没有任何代码会在用户回复“确认”/“好的删掉”之后去调 `apply_adjustment()`——还是原来那句话，`shared_state` 里加一个 `pending_proposal` 字段存最近一次未确认的提案，靠下一轮意图识别或简单的确认/取消关键词触发调用，这部分设计还没定，需要先讨论；也没有把真实预警写进 `weather_alerts` 这一

步（提案制故意不在评估阶段写，见 `exception_agent.py` 一节，`apply_adjustment()` 目前也只删节点、不写 `weather_alerts`，要不要在确认执行时一并写，是留到触发逻辑时一起定的点）

12. ~~两套独立天气数据源~~ 已解决（2026-09-12）：`nearby_sources.weather()` 原来接的是 Open-Meteo，现在改成统一用 `weather_tool.get_hourly_forecast()`（和风天气逐小时预报，免费版最多查 240 小时=10 天），跟 `exception_agent.check_weather()` 用的灾害预警接口共用同一个 key/host。`nearby_sources.summarize_weather()` 的输入契约也跟着换了（从 Open-Meteo 那种 `{"hourly": {"time": [...], "temperature_2m": [...], ...}}` 嵌套结构，改成扁平的 `[{"time": ..., "temperature_c": ..., "rain_probability": ..., "condition_text": ...}]` 列表），判断"是否有雷暴"从匹配 Open-Meteo 的数值天气码（`weather_code >= 95`）改成直接检查和风天气返回的 `condition_text` 里有没有"雷"字，更直观也不用记一张码表。真实限制：预报范围从 Open-Meteo 的 16 天缩到了 10 天，超出范围会走"选定日期不在预报范围内"的降级提示，不会报错崩溃
13. `orchestrate()` 没有从用户消息里提取日期给 `ota_hotel_agent.run()`（2026-09-13 接入真实酒店数据后端到端测试时发现，之前是假数据看不出这个问题）：不管用户说"9月20号"还是别的日期，`booking` 意图命中时传给 `run()` 的 `date_range` 永远是 `None`，酒店查询永远默认"明天起 2 晚"。真实测试里 LLM 最后组句时诚实地发现了这个落差并跟用户说明（防幻觉提示词生效了），但功能上确实没做到"按用户说的日期查"。修法定待，两个方向：简单加一段日期正则/关键词提取；或者扩展 `classify_intent` 顺带做结构化实体抽取（日期/星级/预算这些一起解决），需要先定思路